

MIAE 215: Computer Programming for Engineers

Arduino Lab 1: Software Setup, USB Drivers & Execution Lifecycle

Concordia University · Department of Mechanical, Industrial & Aerospace Engineering

Instructor: Prof. Brandon W. Gordon · Target Microcontroller: Atmel ATmega328P / Arduino Uno R3

1. Laboratory Overview & Objectives

Arduino Lab #1 introduces mechanical, industrial, and aerospace engineering students to embedded computing, physical microcontroller hardware, USB driver toolchains, and real-time execution timing in embedded C++.

LAB 1 CORE LEARNING GOALS
1. Configure the Arduino IDE 2.x environment and USB communication bus
2. Understand the ATmega328P Harvard architecture & memory layout
3. Master the setup() vs. loop() embedded C++ execution lifecycle
4. Identify 8-bit AVR data types (16-bit int, 32-bit float/double)
5. Implement 115,200 baud serial communication with the host PC
6. Measure elapsed time with microsecond accuracy using micros()

[!NOTE] **Academic Policy Reminder:** Attendance in scheduled lab rooms is **completely optional**. If you can complete this lab on your personal computer and verify both example programs, you do not need to attend in person. In-person sessions serve as open support hours with Teaching Assistants (TAs).

2. Hardware Architecture & Technical Specifications

The Arduino Uno R3 is powered by the **Microchip / Atmel ATmega328P**, an 8-bit AVR RISC microcontroller.

ATMEL ATmega328P MICROCONTROLLER
Clock: 16.0 MHz (62.5 ns/cycle) Operating Voltage: 5.0 Volts
HARVARD MEMORY
• Flash ROM : 32 KB (Program Code) • SRAM : 2 KB (Variables/Heap) • EEPROM : 1 KB (Non-volatile)
I/O PINS
• 14 Digital I/O Pins (D0 to D13) - 6 Hardware PWM Pins (~3,5,6,9)

- Built-in LED on Pin 13
- 6 Analog Inputs (A0 to A5, 10-bit)
- USART Hardware Serial (D0/RX, D1/TX)

Critical Data Type Traps on 8-bit AVR (Exam Alert!)

On modern 64-bit desktop PCs, `int` is 32 bits and `double` is 64 bits. **This is NOT true on the ATmega328P:**

Data Type	PC (x86_64)	Arduino Uno (8-bit AVR)	Minimum / Maximum Range on Arduino
<code>int</code>	4 bytes (32-bit)	2 bytes (16-bit)	-32,768 to +32,767 (Exceeding 32,767 overflows to -32,768!)
<code>unsigned int</code>	4 bytes (32-bit)	2 bytes (16-bit)	0 to 65,535
<code>long</code>	4/8 bytes	4 bytes (32-bit)	-2,147,483,648 to +2,147,483,647
<code>float</code>	4 bytes (32-bit)	4 bytes (32-bit)	$\approx 3.4 \times 10^{\pm 38}$ (6–7 digits precision)
<code>double</code>	8 bytes (64-bit)	4 bytes (32-bit)	Identical to <code>float</code> ! There is no 64-bit hardware double precision.

3. Software Environment & Toolchain Setup

Step 1: Download and Install Arduino IDE

1. Download the latest **Arduino IDE 2.x** from the official site: <https://www.arduino.cc/en/Main/Software>
2. Run the installer and accept all USB driver installation prompts.

Step 2: USB Driver Verification

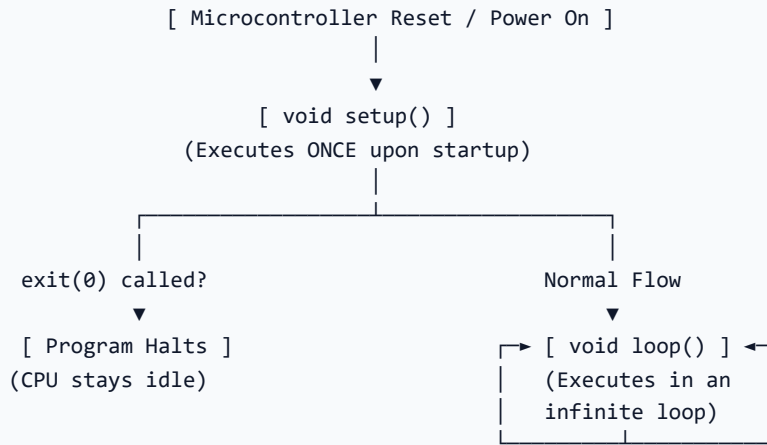
- **Authentic Uno / Mega:** Uses an auxiliary ATmega16U2 chip. Windows and macOS detect this automatically without external drivers.
- **Clone Uno Boards (Elegoo, SunFounder, etc.):** Many third-party boards utilize the **WCH CH340** or **Silicon Labs CP2102** USB-to-UART bridge. If your computer displays an "Unrecognized USB Device" error, download and install the CH340 driver:
 - Windows: Search for "CH341SER.EXE" from WCH.
 - macOS: Download the CH34x macOS driver package.

Step 3: Board & Port Selection

1. Plug the Arduino board into your computer's USB port using the provided USB A-to-B cable.
2. In the Arduino IDE top toolbar, select:
 - **Board:** `Arduino Uno`
 - **Port:**
 - Windows: `COM3` , `COM4` , etc. (Check Device Manager \$to\$ Ports).
 - macOS: `/dev/cu.usbmodem...` or `/dev/cu.usbserial...`

4. The Embedded C++ Execution Lifecycle

Every standard Arduino program requires two fundamental functions:



1. void setup()

- Runs **exactly once** when the board is powered up or the hardware reset button is pressed.
- Used to configure pin modes (`pinMode()`), initialize communication buses (`Serial.begin()`), and allocate variables.
- **Prof. Gordon's Preferred Paradigm:** For linear algorithms and numerical computations, you can place your entire program logic inside `setup()` , followed by `delay(1000)` and `exit(0)` . This turns `setup()` into a standard `main()` function and prevents `loop()` from running!

2. void loop()

- Executes **repeatedly in an infinite loop** after `setup()` completes.
- Used for real-time control, continuous sensor polling, and feedback systems.

5. Lab 1 Example 1: Serial Output & Lifecycle Control

This program demonstrates serial port initialization at **115,200 baud**, variable printing, and using `exit(0)` to halt execution at the end of `setup()` .

Source Code: `arduino_example1_hello_world.ino`

```
// =====
// MIAE 215 - Arduino Lab 1: Example 1 - Hello World & Program Lifecycle
// Instructor: Prof. Brandon W. Gordon · Concordia University
// =====

void setup()
{
    // setup() is analogous to main() in desktop C++

    int i = 1; // 2-byte integer (-32,768 to +32,767)
    // Note: On Arduino Uno, there are no 64-bit doubles -> use floats!
    // On a PC, int is 4 bytes (-2 billion to +2 billion).

    // Initialize the USB Serial monitor tool:
    // 115200 bits/second (baud rate) communication with the PC
    Serial.begin(115200);
}
```

```

// In Arduino, we don't use std::cout; we use Serial.print() instead
Serial.print("\nhello world\n");
Serial.print(i); // How to print a variable value

delay(3000); // Wait 3000 ms = 3.0 seconds

Serial.print("\ngoodbye world\n");

// The exit(0) function stops program execution here,
// preventing loop() from executing.
delay(1000); // Wait 1000 ms to give the serial buffer time to finish transmitting

// exit(0); // Uncommenting this stops the program like pulling the plug!
// -> When you terminate here, setup() behaves identically to main().
// -> Your intuition about C++ main() applies directly to setup().

} // End of setup function

// The loop() function executes repeatedly as fast as possible
// after setup() completes -- like an infinite while(true) loop.
void loop()
{
    Serial.print("\nhello world -- loop\n");
    delay(500); // Repeat every 500 ms
}

```

Expected Output in Serial Monitor (115200 Baud):

```

hello world
1
goodbye world
hello world -- loop
hello world -- loop
hello world -- loop

```

[!IMPORTANT] **Serial Monitor Baud Rate Match:** Make sure the dropdown at the bottom right corner of the Arduino Serial Monitor window is set to **115200 baud**. If it is left at the default 9600 baud, the output will appear as scrambled, unreadable characters!

6. Lab 1 Example 2: Real-Time Clock & Microsecond Timing

This program demonstrates how to read the hardware microsecond timer using `micros()`, convert elapsed time to seconds, and format high-precision floating-point output.

Source Code: `arduino_example2_clock_function.ino`

```

// =====
// MIAE 215 - Arduino Lab 1: Example 2 - Real-Time Clock Function
// Instructor: Prof. Brandon W. Gordon · Concordia University
// =====

```

```

void setup()
{
    float t; // Time variable in seconds

    Serial.begin(115200);
    Serial.print("\nsetup complete");

    // To measure time in microsecond intervals, call micros().
    // Returns the number of microseconds elapsed since program startup.
    // (i.e., t = 0 when the ATmega328P begins executing).
    t = micros() * 1.0e-6; // Convert microseconds to seconds

    Serial.print("\nt = ");
    Serial.print(t);

    // Continuous polling loop using an infinite while loop:
    while(1)
    {
        t = micros() * 1.0e-6; // Update elapsed time in seconds
        Serial.print("\nt = ");
        Serial.print(t, 7);    // Print float with 7 decimal places
        delay(100);            // Delay approximately 0.1 s (100 ms)
    }

    // Note: The reason for extra decimal digits drifting (e.g., 0.10017, 0.20121)
    // is that delay() is not perfectly deterministic -- it can be interrupted
    // by timer interrupts and serial transmissions.

    delay(1000);
    exit(0);
}

void loop()
{
    // loop() is never reached because the while(1) loop in setup() never terminates.
}

```

Technical Timing Insights:

1. **micros() Resolution:** Increments in steps of **4 microseconds** on a 16 MHz Arduino Uno (due to the 64 prescaler on Timer0).
2. **micros() Rollover:** The 32-bit unsigned integer counter overflows and resets to zero after approximately **70 minutes** ($2^{32} \mu\text{s} \approx 4,294.96 \text{ seconds} \approx 71.58 \text{ minutes}$).
3. **delay() Limitation:** The `delay(100)` function blocks the CPU and suffers from timing jitter. For precise mechatronic sampling (e.g., in Lab 2 and the Term Project), students will use non-blocking timestamp delta checks:

```
if (micros() - last_time >= target_period) { ... }
```

7. Self-Study Verification Checklist

Before moving to **Lab 2 (Week 6)**, ensure you have verified:

- [] Arduino IDE 2.x connects to your Uno board without port permission errors.

- [] `arduino_example1_hello_world.ino` uploads and displays readable text on the Serial Monitor at 115200 baud.
- [] `arduino_example2_clock_function.ino` continuously streams timestamp values to the console with accurate second intervals.
- [] You understand that on the ATmega328P, `int` is **16 bits** (`-32,768` to `+32,767`) and `double` is identical to `float` (**32 bits**).